

REWEAR-FUSED — The No-Limits Frontend Frontier Report (June 2026)

TL;DR

- *The maximum-ceiling stack for REWEAR-FUSED is a single WebGPU/TSL pipeline (three r184 + WebGPURenderer, fully production after Safari 26's Sep 2025 ship) driving (a) Mol 5.9.0 in SSGI "illumination mode" with weighted-blended + dual-depth-peeling OIT for the enzyme and polymer, (b) TSL compute shaders generating 1M+ GPU particles, SDF/ray-marched carbamate-bond animations, and marching-cubes molecular surfaces, (c) three-gpu-pathtracer for offline-grade hero stills, and (d) Gaussian-splat photoreal "garment-to-monomer" transition scenes* — all orchestrated by React 19.2 with `<Activity>`-driven screen preloading, Lenis 1.3.23 + GSAP 3.13+ cinematic scroll, deck.gl v9 / luma.gl WebGPU charting, and an in-browser WebLLM/Transformers.js v4 co-pilot.*
- **What makes it "shouldn't be possible on the web":** the same TSL nodegraph (`three/tsl`) compiles to both WGSL and GLSL, so one source drives a path-traced enzyme, a million-particle textile-fiber recycling cyclone, a real-time SDF carbamate-cleavage animation, and an SSGI-lit molecular dashboard — at 60-120 fps on the ~82.7 % of users on Baseline WebGPU browsers (per buildmvpfast.com, citing the Nov 25 2025 Baseline milestone; caniuse shows ~87 % desktop / ~71 % mobile in Feb 2026), with automatic WebGL2 fallback for the rest at zero developer cost. ([LearnWithHasan](#)) ([BuildMVPFast](#))
- **Concrete frontier moves to make on day one:** import `three/webgpu` not `three`, write everything in TSL not GLSL, drive Mol* with a MolViewSpec (`.mvsj`) timeline (animation + snapshot transitions are native in MVS v1.8.1), turn on Mol*'s (`?illumination=1`) SSGI + (`plugin-config.transparency=dpoit`), pre-render hidden screens inside React 19.2 (`<Activity mode="hidden">`), and ship a local Qwen 2.5 1.5B or Phi-3.5-mini "molecular co-pilot" via ([@huggingface/transformers](#)) v4 on WebGPU.

Key Findings

1. **WebGPU is Baseline as of November 25 2025.** Chrome 113+, Edge 113+, Firefox 141 (Windows) / 145 (ARM macOS), Safari 26 (macOS Tahoe / iOS 26 / iPadOS 26 / visionOS 26). Per buildmvpfast.com (which cites the Chromium Dev announcement), the November 25 2025 Baseline milestone "jumped [global coverage] to about 82.7%"; per TheBomb.ca summarizing caniuse in February 2026, "global support sits at roughly 87% on desktop and 71% on mobile, with iOS Safari the main laggard on older devices." Automatic WebGL2 fallback handles the rest. For REWEAR-FUSED, **WebGPU is the primary renderer, not the aspirational one** — there is no longer a reason to default to WebGL2. ([VideoCardz](#))

2. **Three.js r184 (March 2026)** ships TSL as the first-class shading API, with one-line WebGPU migration (`import * as THREE from 'three/webgpu'`). r184 specifically adds **FSR 1 for WebGPURenderer**, a **TSL Graph Addons** system, a **Timeline Commands Recorder**, **partial buffer updates**, a **stack-trace inspector**, and SSGI ball-pool / FPS marching-cubes WebGPU demos. Per-frame object allocations were eliminated to stop the multi-hundred-thousand-objects/sec GC pressure that 1000-mesh scenes used to cause. [AlterSquare](#)
3. **Mol 5.9.0 (May 3 2026)*** is the publication-grade ceiling for the enzyme and polymer scenes. Critical frontier features confirmed:
- **Screen-Space Global Illumination (SSGI)** "illumination mode" — path-traced, progressively sampled, adaptive-denoised; toggled by pressing G, the `?illumination=1` URL param, the viewport button (added v5.4.1), or per-snapshot via MVS `canvas.custom.molstar_postprocessing`. Verbatim from molstar.org/illumination/: *"Our screen-space global illumination (SSGI) provides natural, balanced lighting by approximating indirect light in screen space... Samples are traced iteratively for interactivity, with adaptive denoising and automatic sample-count adjustment to maintain consistent performance across GPUs."* [github](#) [molstar](#)
 - **Order-independent transparency with three modes** exposed via `plugin-config.transparency`: `blended` (sorted), `wboit` (McGuire-Bavoil weighted-blended OIT), and `dpoit` (dual depth peeling). v5.0.0 added a **Monolayer transparency** mode that exploits dpoit. Mol*'s own viewer docs: *"WBOIT stands for 'weighted blended order-independent transparency' [McGuire & Bavoil, JCGT 2013]. It allows more accurate rendering of overlapping transparent objects at the expense of some speed."* [GitHub](#) [Mol*](#)
 - **GPU procedural animation new in 5.9.0**: per-vertex wiggle via fBm noise, per-instance tumble (rotation+translation), `Wiggle` data-driven theme, `enableAnimation` Canvas3D param — exactly the "breathing protein" cinematic effect, free, no custom shader. [github](#)
 - **8K UHD `ViewportScreenshotHelper`** native — publication-grade stills in-app. [github](#)
 - **MolViewSpec (MVS) v1.8.1** lets you script the entire enzyme story as a single `.mvs.j` JSON tree (Component, Representation, Color, Label, Tooltip, Camera, Focus, Primitives ellipse/box/arrow/mesh/sphere, Annotations, transform.rotation_center, `Snapshot.animation` animating "almost every property", `snapshot_key` click-to-transition). The `molstar_postprocessing` custom property toggles outline, DoF, bloom, shadow, SSAO, fog, background per scene. [GitHub](#) [Mol*](#)
 - **Volumes & Segmentations (Mol*VS 2.0)** streams cryo-EM / EM-density / OME-NGFF / EMDB-SFF .hff / .ccp4 / .mrc / .map / Amira / Segger volumes interactively from a server backend.
 - **Mesoscale Explorer** renders **3.07-billion-atom** Petworld synaptic-bouton scenes at interactive rates via LOD, GPU instancing, frustum + occlusion culling, and SSGI.

- Mol* itself is **still WebGL2** (with WebGL1 fallback) — WebGPU is on the long-term roadmap (molstar.org embeds the Web3DSurvey collector to track WebGPU adoption), but the project shipped a `ShaderManager` with `KHR_parallel_shader_compile` parallel compilation in 5.0.0 instead. [Google Groups + 2](#)
4. **GPU pathtracing on the web** is production-shippable: `three-gpu-pathtracer` (gkjohnson) with the `WebGLPathTracer` class, `PhysicalCamera` (focusDistance, fStop, bokehSize, apertureBlades, anamorphic ratio), `PhysicalSpotLight`, `BlurredEnvMapGenerator`, IES profiles, multi-importance sampling, and ACES Filmic tone-mapping — wrapped in `@react-three/gpu-pathtracer` 0.3.x `<Pathtracer minSamples={5} :antCitation[] {citations="03022d6d-2249-4b2b-934b-3032ba71db43" injected="space"} bounces={6} tiles={[2,2]}>`. [GitHub + 2](#)
 5. **GPU compute in TSL** puts millions of particles, marching-cubes molecular surfaces, and SDF/raymarched metaballs on the GPU with one `three/tsl` import. `instancedArray()`, `storage()`, `Fn(() => {...})`, `compute()`, `renderer.computeAsync(...)` are the four APIs you need. Reference production: Utsubo's "**Waves of Connection**" interactive installation at Expo 2025 Osaka — a Hokusai Great Wave piece running Three.js WebGPU 1 M particles on a 98-inch screen with Kinect skeleton tracking, logging 10,000+ visitor interactions over 7 days (utsubo.com/blog/hokusai-interactive-installation). Will Usher's WebGPU data-parallel marching cubes (willusher.io, 2024) is the reference compute pipeline; Twinklebear/webgpu-bcmc adds block-compressed terascale isosurface extraction. [Utsubo + 3](#)
 6. **React 19.2 (Oct 1 2025)** gives the frontend three weapons specifically suited to a heavy 3D + scrollytelling app:
 - `<Activity mode="hidden">` pre-renders unmounted screens in the background while preserving state — the molecular co-design viewer, the closed-loop animation, the passport dashboard, and the pitch narrative all warm-load in parallel without firing effects until visible. Per react.dev/blog: *"hidden: hides the children, unmounts effects, and defers all updates until React has nothing left to work on... You can use Activity to render hidden parts of the app that a user is likely to navigate to next, or to save the state of parts the user navigates away from."*
 - **Suspense batching** in SSR groups multiple boundaries' reveals so the page doesn't pop element-by-element.
 - `useEffectEvent`, `cacheSignal`, **partial pre-rendering** (`prerender` → `resume`), and `<ViewTransition>` SSR support.
 - The **React Compiler** eliminates the `useMemo`/`useCallback` tax — important because R3F scenes are full of stable refs that would otherwise re-render on every scroll tick.
 7. **Award-winning patterns to clone (Awwwards 2025/2026)** that map onto REWEAR-FUSED's screens:

- **Lando Norris** (OFF+BRAND, Awwwards Site of the Year 2025) — Webflow + WebGL + Rive cinematic scroll → editorial typography model for the passport dashboard. (Futurists) (Awwwards +2)
 - **Messenger** (Awwwards Site of the Year 2025 / Developer Site of the Year) — WebGL planet with physics → analog: the closed-loop circularity globe. (Awwwards +2) (Spinxdigital)
 - **Bruno Simon 2025** (Site of the Month Jan 2026) — drivable WebGL world with Cannon.js physics → analog: drivable inspection of the recycled-fiber loop. (Awwwards + 2)
 - **Samsy** — cyberpunk WebGPU portfolio at 120+ FPS, holographic HUD.
 - **OHZI Interactive** — cursor-reactive custom GLSL shaders.
 - **Ameen Abdullah** — WebGPU sakura petal compute particle loader → analog: TSL compute fiber-particle loader. (Creativedevjobs)
 - **Lusion Gemini** — single Three.js model rendered in two visual modes with a slow-motion cinematic HUD on hold → exact pattern for "molecular viewer ↔ industrial viewer" toggle.
 - **Active Theory** (12-year Awwwards veteran, custom Hydra framework) — scroll-driven cross-world WebGL transitions with baked 500K-poly meshes. (Webgpu)
8. **In-browser AI is real, fast, free per-inference:** WebLLM (@mlc-ai/web-llm), OpenAI-compatible API, 17.6 k stars) hits 41 tok/s on Llama 3.1 8B INT4 and 71 tok/s on Phi-3.5-mini on Apple M3 Max. Per Hugging Face's official Transformers.js v4 release blog (huggingface.co/blog/transformersjs-v4, Feb 2026): *"In our tests, we've been able to run GPT-OSS 20B (q4f16) at ~60 tokens per second on an M4 Pro Max."* Chrome's Gemini Nano is exposed as a zero-download Built-in AI API. (BuildMVPFast + 2)
9. **GSAP 3.13+** is fully free including ScrollSmoother / SplitText / ScrambleTextPlugin / MorphSVG / MotionPath, since the GSAP 3.13 release on **April 29 2025** (per gsap.com/blog/3-13 and confirmed in production guides — minimum version for all free bonus plugins is 3.13, not 3.12). **Lenis 1.3.23** (Darkroom Engineering) with (autoRaf: true, autoToggle: true, anchors: true, allowNestedScroll: true, :antCitation[] {citations="98adc831-0fb8-4cc6-bf00-a14d8c10ef2c" injected="space"} stopInertiaOnNavigate: true) is the recommended scroll engine, synced to GSAP via (gsap.ticker.add(time => lenis.raf(time * 1000)) + (gsap.ticker.lagSmoothing(0)). (GitHub)
10. **deck.gl v9 / luma.gl v9** has WebGPU support (alpha) — GPU-aggregated heatmaps, GridLayer/HexagonLayer with (gpuAggregation: true), and ChartGPU-class 1 M-point real-time rendering at 60 fps. Pair with visx 3.12 / Observable Plot / D3 v7 for editorial small-multiples.
-

Details

PART 1 — The Bleeding-Edge 3D / GPU Web Rendering Frontier

(a) WebGPU as the primary renderer. Per the W3C GPU for the Web Working Group's Nov 25 2025 Baseline announcement, WebGPU now ships by default everywhere. The Three.js manual (threejs.org/manual/en/webgpurenderer.html) describes `WebGPURenderer` as the new default and explicitly states `WebGLRenderer` is "still maintained" but has "no plans to add larger new features." The migration is one line: `Three.js`

```
js
- import * as THREE from 'three';
+ import * as THREE from 'three/webgpu';
const renderer = new THREE.WebGPURenderer({ antialias: true });
await renderer.init(); // mandatory in r184
```

Default to WebGPU; allow `forceWebGL: true` only for the e2e screenshot test pipeline.

Automatic WebGL2 fallback covers the ~13 % desktop / ~29 % mobile users without WebGPU.

Use the new **TSL post-processing stack** (`PostProcessing` class, not `EffectComposer`):

`outputNode = scenePass.pipe(bloom()).pipe(fxaa())`. R3F v9's async `gl` prop factory is the React-side hook.

(b) TSL (Three Shading Language) at the frontier. Authored in JavaScript, compiles to WGSL + GLSL from one source. The minimal pattern for the cleavable-bond shader on the polymer is:

```
js
import { uniform, time, sin, positionLocal, normalLocal, vec3, mx_noise_float } from
import { MeshStandardNodeMaterial } from 'three/webgpu';
const cleavageProgress = uniform(0.0);
const wave = sin(positionLocal.x.mul(8.0).add(time)).mul(cleavageProgress);
const displaced = positionLocal.add(normalLocal.mul(wave.mul(0.3)));
const mat = new MeshStandardNodeMaterial();
mat.positionNode = displaced;
mat.colorNode = vec3(0.16, 0.85, 0.55).mix(vec3(0.95, 0.34, 0.21), cleavageProgress)
```

Compute shaders use `instancedArray()`, `storage()`, and `Fn()` from `three/tsl`. The canonical 1 M-particle pattern is documented at utsubo.com/blog/webgpu-threejs-migration-guide and threejsroadmap.com.

(c) GPU compute for molecular visualization.

- **Millions of particles:** GPGPU particles with TSL (Wawa Sensei "GPGPU particles with TSL & WebGPU"; Maxime Heckel's "Field Guide to TSL and WebGPU"). Use for the textile-fiber recycling cyclone, monomer dust cloud, and digital-passport data-rain background.
- **Marching cubes on the GPU:** Will Usher's WebGPU data-parallel marching cubes (willusher.io/graphics/2024/04/22/webgpu-marching-cubes/) is the reference data-parallel implementation with stream compaction, bind-group dynamic buffer offsets, and near-native Vulkan performance on RTX 3080. Twinklebear/webgpu-bcmc adds block-compressed terascale isosurface extraction; threejs.org ships `(webgpu_tsl_compute_attractors_particles)` and `(webxr_xr_marchingcubes)` examples in r184. Use this to compute **solvent-excluded surfaces (SES) and electron-density isosurfaces on the GPU at interactive rates** as the user changes contour level. (Will Usher)
- **SDF / ray-marching molecular surfaces:** Codrops "Liquid Raymarching with TSL" (2024) and Codrops "Droplet-like Metaballs with Three.js" (June 2025) cover SDF metaballs implemented in TSL — adapt the metaball blend (`(smoothMin)`) directly to atom positions for an alternative SES/Gaussian-surface look that needs no mesh.
- **Real-time electron-density isosurfaces:** Mol* already ships this for cryo-EM via Volumes & Segmentations; layer GPU marching-cubes on top of pre-computed CCP4/MRC density grids served statically.
- **Mass-spring cloth at scale:** per "Real-Time Cloth Simulation Using WebGPU: Evaluating Limits of High-Resolution" (Sung, Ma, Kim, Choi, Choi, Hong, Soonchunhyang University et al., arXiv:2507.11794, July 15 2025), the WebGPU mass-spring method maintains "60 frames per second (fps) even with up to 640K nodes." This is the model for the polymer-chain reptation simulation. (arXiv + 2)

(d) Advanced rendering techniques.

- **Path tracing:** `(three-gpu-pathtracer)` (gkjohnson) provides `(WebGLPathTracer)` with `(PhysicalCamera)`, `(PhysicalSpotLight)`, `(BlurredEnvMapGenerator)`, IES profiles, multiple-importance sampling. Wrap in `<Pathtracer minSamples={5} bounces={6} tiles={{[2,2]}}>` from `@react-three/gpu-pathtracer`. Use ACES Filmic tone mapping. Render the **carbamate-bond hero still and the enzyme-active-site close-up** as path-traced converged samples while the user reads.
- **Gaussian splatting:** `(@mkkellogg/gaussian-splats-3d)` (with `(gpuAcceleratedSort: true)`) and `(zappar-xr/three-gaussian-splat)` give R3F integration. Used for the **garment-to-monomer photoreal transition scene**: capture a real child's stretch garment as a Gaussian splat, dissolve it into the molecular world. Requires `(Cross-Origin-Embedder-Policy: require-corp)` + `(Cross-Origin-Opener-Policy: same-origin)` headers; serve `(.splat)` / `(.ply)` / `(.ksplat)`. (GitHub)
- **Screen-space effects:** r184 ships TSL implementations of bloom, FXAA, SMAA, SSAO, SSR, SSGI, SSS, and a "better DoF exclusively for the new renderer" (threejs.org/manual).

Layer SSGI + bloom + DoF + chromatic aberration + film grain + vignette on every screen.

[Three.js](#)

- **Order-independent transparency:** Mol* exposes three modes via `plugin-config.transparency`: `blended` | `wboit` | `dpoit`. Per Mol*'s viewer-docs query parameters: *"WBOIT stands for 'weighted blended order-independent transparency'... It allows more accurate rendering of overlapping transparent objects at the expense of some speed."* Set Mol*'s transparency to `dpoit` for the publication-grade enzyme surface stack; v5.0.0 added a **Monolayer transparency** that specifically exploits `dpoit`. [GitHub + 2](#)

(e) Mol 5.9 pushed to the maximum.*

- **Illumination mode (SSGI):** enable via URL param `?illumination=1`, key `G`, the viewport button (added v5.4.1), or MVS `canvas.custom.molstar_postprocessing`.
- **MolViewSpec (MVS) v1.8.1:** drive the entire enzyme narrative as a single `.mvsj` JSON tree. Per [molstar.org/mol-view-spec-docs/](#), MVS v1 building blocks are: *"Component, Representation, Color, Label, Tooltip, Camera, Focus, Annotations, Primitives, Multiple States & Animations."* Mol* 5.0 expansion added (verbatim): *"Generic color schemes (palette parameter for color_from_nodes); Annotation field remapping; representation node: support custom property `molstar_representation_params`; Add `backbone` and `line` representation types; primitives node: support custom property `molstar_mesh/label/line_params`; canvas node: support custom property `molstar_postprocessing` with the ability to customize outline, depth of field, bloom, shadow, occlusion (SSAO), fog, and background; clip node support for structure and volume representations; grid_slice representation support for volumes; Support `snapshot_key` parameter on primitives that enables transition between states via clicking on 3D objects; Inline selectors and MVS annotations support `instance_id`; Add `transform.rotation_center` property that enables rotating an object around its centroid or a specific point; Support transforming and instancing of structures, components, and volumes; Add `Snapshot.animation` support that enables animating almost every property in a given tree."** The file is `.mvsj` (JSON) or `.mvsx` (ZIP archive containing `index.mvsj` + referenced assets). [github](#)
- **GPU procedural animation (new in 5.9.0):** per-vertex wiggle, per-instance tumble, `Wiggle` theme, `enableAnimation` Canvas3D param. The Viewer ships a "Procedural Animation panel" out of the box. This is your "living enzyme" effect — free, no custom shader. [github](#)
- **Custom representations / Behaviors:** register a `StateTransformer.builderFactory('rewear-fused')` for custom data → Representation3D nodes; the canonical pattern (molstar issue #146) builds a `LabelRepresentation` from `plugin.canvas3d?.webgl` and `plugin.representation.structure.themes`. The 5.8 release moved debug helpers to a

`PluginBehavior` pattern (params no longer on Canvas3D; helpers for clip-object, direct-volume, image, mesh). [github](#) [github](#)

- **Loading-extension hook:** `MolstarLoadingExtension<>` lets you react to MVS nodes during load (e.g., toggle `isHidden`, inject custom geometry). [Mol*](#)
- **8K UHD `ViewportScreenshotHelper`** (5.9.0) — render publication-grade stills in-app. [github](#)
- **`ShaderManager` with `KHR_parallel_shader_compile`** (5.0.0) — adaptive parallel shader compilation, Scene only draws when shaders are ready. [github](#)

(f) PBR, HDR, bloom, DoF, chromatic aberration, film grain. All in the TSL post-processing stack. Use UltraHDR JPGs via `UltraHDRLoader` (`HDRJPGLoader` is removed) and `RoomEnvironment` PMREM. The cinematic look comes from layering: SSGI base → emissive bloom on the catalytic residues → tight-aperture DoF (`fStop ~1.4`, `bokehSize ~30`, `apertureBlades = 6`) → subtle chromatic aberration on viewport edges → grain (Perlin/blue-noise) → vignette. [GitHub](#)

PART 2 — The Most Advanced React / Web Architecture Frontier

(a) React 19.2. Default to the **App Router with Server Components** in Next.js 16.2; mark every R3F canvas with `"use client"`; pass all static artifacts (PDB/CIF/JSON/MVS) as Server-Component-fetched data with the `cache(async fn)` pattern. Use `<Activity mode="hidden">` to pre-mount the four major screens so navigations feel instant and state (camera position, selections, hover history) persists across visits. Use `use(promise)` inside Suspense boundaries to read pre-computed static artifacts as if synchronous; `useEffectEvent` for camera-control event handlers that must read fresh state without re-binding; `cacheSignal` for shared GPU resource caches across screens. Use **partial pre-rendering** (`prerender()` → `resume()`) so the editorial copy is on the CDN and the live 3D viewer streams in.

(b) State orchestration. Zustand 4+ for global UI state (selected residue, scroll progress, AI co-pilot session), Jotai for derived atoms (e.g., "currently active MVS snapshot" derived from scroll), and the React Compiler removes all `useMemo`/`useCallback`. The R3F ecosystem at the frontier: `@react-three/fiber` v9 (async gl prop), `@react-three/drei` (Preload, Detailed LOD, Environment, OrbitControls, Html), `@react-three/postprocessing` (WebGL fallback), native `three/tsl` post-processing (WebGPU), `@react-three/rapier` for physics, `@react-three/gpu-pathtracer` 0.3.x for path-traced hero renders, `r3f-perf` and `stats-gl` for profiling, `leva` for live param tweaking (dev-only). [Creativedevjobs](#)

(c) Performance engineering at the limit.

- **OffscreenCanvas + Worker rendering:** ship the cinematic background canvas to a worker via `canvas.transferControlToOffscreen()` so the main thread is free for React + Mol* + AI.
- **SharedArrayBuffer + multi-threaded WASM:** required headers `Cross-Origin-Embedder-Policy: require-corp` + `Cross-Origin-Opener-Policy: same-origin` (already needed for Gaussian splatting and Transformers.js multi-thread).
- **WASM (Rust→WASM):** pre-compile any in-browser PDB/CIF parser or compliance-rule evaluator to WASM with SIMD; move rule evaluation (e.g., "does this monomer contain a banned substance?") to a Rust WASM worker.
- **AssemblyScript** is the lighter alternative for the compliance-rule DSL.

(d) The 60-120 fps architecture.

- React Compiler eliminates re-render tax on R3F scenes.
- Mutate Three.js objects directly inside `useFrame`, never via React state.
- One Mol* canvas per screen, but **share a single Three.js WebGPURenderer** for everything else via R3F's `Canvas` and `useThree(state => state.gl)`.
- `renderer.info.render.calls` < 100 target; use `BatchedMesh` / `InstancedMesh` for fiber chains; KTX2 + Meshopt for textures.
- `<Activity hidden>` for pre-mounted screens; `<Detailed>` LOD for the molecule (atomic → coarse → glyph at distance).
- `r3f-perf` and `stats-gl` overlays in dev.

PART 3 — Cinematic / Motion / Scrollytelling Frontier

(a) **Scrollytelling architecture.** Lenis 1.3.23 with `autoRaf: true, autoToggle: true, anchors: true, allowNestedScroll: true, stopInertiaOnNavigate: true` + GSAP 3.13+ ScrollTrigger via `gsap.ticker.add(time => lenis.raf(time * 1000))` and `gsap.ticker.lagSmoothing(0)`. Scroll progress drives:

1. Mol* camera via `plugin.canvas3d.camera.setState({ position, target, ...})` interpolated by GSAP.
2. MVS `Snapshot.animation` timeline scrubbed by scroll progress.
3. TSL uniform `scrollProgress` directly into the cleavage shader.
4. Post-processing uniforms (bloom intensity, DoF focal distance, vignette strength).

(b) Award-winning techniques to clone.

- **OFF+BRAND / Lando Norris pattern** (Awwwards Site of the Year 2025): bold lime-green editorial typography + scroll-driven 3D rotation + Rive UI overlays — applied to the Digital

Recyclability Passport landing. (Awwwards)

- **Active Theory** (Hydra framework, 12 years of Awwwards): scroll-driven WebGL worlds with cross-world fluid transitions, baked 500 K-poly meshes to mobile-friendly LOD. (Webgpu)
- **Lusion Gemini** (London/Bristol): "click and hold → slow-motion + cinematic HUD" via GSAP — applied: hold-to-zoom into the catalytic pocket, time slows, HUD shows residue identity / energetics.
- **basement.studio / unseen-studio / Metabole**: editorial type-in-motion, custom fluid cursors, fluid scroll snaps.
- **Bruno Simon 2025** (Awwwards Site of the Month Jan 2026): driveable physics scene — for REWEAR-FUSED, a driveable circularity loop with Cannon.js / Rapier physics. (Awwwards) (Spinxdigital)

(c) **Advanced shader-driven transitions.** Custom GLSL/TSL page transitions (RGB-shift dissolve, voronoi reveal, fluid simulation transitions), generative particle backgrounds (TSL compute fBm noise), fluid simulations (WebGPU mass-spring cloth at 640 K nodes / 60 fps per arXiv:2507.11794), custom magnetic cursors (Codrops standard pattern), shader-driven SplitText character reveals.

(d) **"Molecular instrument meets editorial science journal."** Editorial New (display) + Söhne / General Sans (body) + JetBrains Mono (data, residue IDs); variable fonts for kinetic typography on scroll; OpenType (ss01) + (tnum) for tabular figures in the dashboard; SplitText (free in 3.13+) animates per-character on every section reveal; ScrambleTextPlugin (free) for the AI co-pilot streaming response.

PART 4 — Data Visualization & Scientific-UI Frontier

(a) Tools.

- **deck.gl v9 / luma.gl v9** for the screening-funnel and trade-off curve at >100 K points; GPU aggregation via (gpuAggregation: true) on HexagonLayer / GridLayer; WebGPU alpha backend with (webgpuAdapter) from (@luma.gl/webgpu). (deck.gl)
- **visx 3.12 / D3 v7 / Observable Plot** for editorial small multiples in the passport dashboard.
- **regl** for custom GPU charts (e.g., a 1 M-cell compliance heatmap).
- **ECharts 6.1** as the fast default for cartesian plots.

(b) **3D ↔ data fusion.** Drei (<Html occlude>) to attach annotations to 3D residues; for performance, render labels in Three.js via TextureAtlas + SDF text (drei (<Text>)). The MVS (Label) and (Tooltip) nodes plus (molstar_mesh/label/line_params) give native pipelines for 3D-anchored annotations on the molecule.

(c) **Real-time / animated viz.** All chart transitions on Motion 12.40 or GSAP for synchronized scroll. The "instrument" feel comes from: monospaced live-updating numbers (JetBrains Mono with `tabular-nums`), thin 1px hairline strokes, sparklines on every KPI, animated SVG ticks, and `<ViewTransition>` (React 19.2) between dashboard states.

PART 5 — Integration & "Shouldn't Be Possible" Ceiling

(a) The integrated experience.

- A **single shared three/webgpu renderer** for everything except Mol* (which keeps its own WebGL2 canvas).
- A **scroll-driven MVS timeline** for the enzyme story.
- A **TSL compute shader cluster** (cleavage animation + particle cyclone + GPU-marching-cubes surfaces) sharing storage buffers.
- A **path-traced converged hero** swapped in when scroll velocity ≈ 0 for >800 ms.
- A **Gaussian-splat garment scene** dissolving into a particle cloud that becomes the polymer chain.
- An **AI co-pilot drawer** running WebLLM (Qwen 2.5 1.5B or Phi-3.5-mini, INT4, WebGPU backend) streaming answers on hover/click of any residue, monomer, or compliance flag.

(b) **AI in the frontend.** Use **WebLLM** (`@mlc-ai/web-llm`) for chat ("Why is this residue catalytic?", "What does REACH SVHC mean?"). Use **Transformers.js v4** (`@huggingface/transformers`) with `device: 'webgpu'` and mixed-precision dtype `{embed_tokens: 'fp16', lm_head: 'fp32', default: 'q4'}` for embeddings + classification on the compliance ledger; per the Hugging Face Transformers.js v4 release blog (Feb 2026): *"we've been able to run GPT-OSS 20B (q4f16) at ~60 tokens per second on an M4 Pro Max."* Use **Chrome's built-in Gemini Nano** as the zero-download fallback for one-line summaries. Use the OpenAI-compatible WebLLM API so the same code can route to a server fallback on integrated-GPU users. Cache models in IndexedDB via the Cache API. Initialize behind a "Capability Check" → "Download with progress" UX. (Maddevs) (Hugging Face)

(c) Most advanced version of each REWEAR-FUSED screen.

1. Molecular Co-Design Viewer (the enzyme + the cleavable elastane).

- Mol* 5.9.0 plugin instance with `?illumination=1`, `plugin-config.transparency=dpoit`, `enableAnimation=true`, Wiggle theme on the enzyme body.
- MVS `.mvsj` tree driving: (i) cartoon for the enzyme with the catalytic triad in ball-and-stick + emissive bloom; (ii) Gaussian molecular surface with dpoit; (iii) primitives (arrows,

ellipses) for the carbamate substrate pose; (iv) custom `molstar_postprocessing` with DoF + bloom on the active site; (v) `Snapshot.animation` cycling apo → bound → cleaved.

- *Beside Mol*, a separate R3F canvas* hosts the polymer-chain view (200-mer elastane chain as instanced cylinders + spheres, GPU procedural wiggle via TSL fBm noise, the cleavable carbamate bond glows and shears when scroll hits the cleavage moment).
- Live linked overlay: hovering a residue in Mol* highlights the matching feature in the polymer chain and triggers a WebLLM annotation.
- "Hold to inspect" GSAP slow-motion + HUD (Lusion Gemini pattern): residue identity, pKa, hydrophobicity scale, ConSurf conservation.

2. Closed-Loop Circularity Animation.

- Full-screen TSL compute shader: 1 M+ particles in three states (garment → monomer → polymer) on a Thomas/Lorenz attractor reshaped each scroll section.
- A driveable physics layer (Rapier or Cannon) with a small inspection vehicle (Bruno Simon homage) that rolls around the recycling loop.
- Gaussian-splat garments at the entry and exit nodes of the loop, dissolving into TSL particles via a custom dissolve shader.
- Camera follows a GSAP `MotionPathPlugin` along a 3D Bezier through the loop.
- WebGPU compute path simulates polymer chain reptation as a mass-spring system (per arXiv:2507.11794, 640 K-node 60-fps capability).

3. Digital Recyclability Passport Dashboard.

- Editorial typography header (Editorial New + JetBrains Mono) + a Lando-Norris-style hero rotating 3D passport "card" (a custom TSL shader on a thin slab with chromatic aberration and SSGI).
- deck.gl WebGPU heatmap of compliance flags across the supply chain (`HexagonLayer` with `gpuAggregation: true`).
- visx + Observable Plot small-multiples for REACH/SVHC, CPSIA, EU Eco-Design.
- WebLLM in a side drawer summarizes each flag in plain English; ScrambleTextPlugin animates the streaming response.
- `<ViewTransition>` (React 19.2) between detail levels so dashboard expansion is one continuous animation.
- Each KPI card hosts a Mol* mini-canvas with the relevant substructure highlighted (one shared Mol* canvas reused via cached MVS snapshots, not one canvas per card).

4. Cinematic Pitch Narrative.

- Pure scroll-driven editorial composition with Lenis + ScrollSmoother + ScrollTrigger.

- Hero open: WebGPU sakura-style compute particle loader (Ameen Abdullah homage), but the petals are elastane fiber fragments.
- Section 1: a Gaussian-splat child's leggings dissolves into TSL compute particles that arrange into a polymer chain.
- Section 2: the polymer chain zooms to atomic scale; Mol* fades in via a shared layered canvas; the enzyme docks in MVS-driven animation.
- Section 3: the carbamate bond cleaves (TSL shader + GSAP scrub); a path-traced converged still locks in.
- Section 4: the chain disassembles back to monomers (TSL particle reversal); the closed loop closes.
- Section 5: the passport dashboard rises from below as a layered editorial spread.
- Section 6: the AI co-pilot welcomes the user; WebLLM streams a 3-sentence "what you just saw" in their language (Transformers.js v4 multilingual embedding chooses).
- Persistent floating HUD: scroll progress, FPS (r3f-perf in dev, hidden in prod), and a "press G for illumination mode" hint.

(d) Awwwards-winning techniques mapped into REWEAR-FUSED.

- **Sub-second LCP** via Server-Components-rendered editorial shell + Activity-preloaded heavy canvases.
- **Custom magnetic cursor** that becomes the active-site probe inside Mol*.
- **WebGPU loader** (TSL compute particles before any JS-heavy work).
- **Cinematic scroll** with GSAP CustomEase curves; one continuous experience, no page reloads.
- **WebGPU sakura-style** petal compute particle reveals between sections.
- **Hold-to-slow-motion** cinematic HUD on the enzyme (Lusion Gemini).
- **Driveable physics loop** (Bruno Simon).
- **AI co-pilot streaming kinetic typography** (ScrambleTextPlugin + SplitText).
- **Editorial typography** with variable-font weight modulation on scroll.

Recommendations

Stage 1 (Frontier baseline, the no-limits floor). Adopt: Next.js 16.2 + React 19.2 + R3F v9 + `(three/webgpu)` r184 + Tailwind v4.3 + Mol* 5.9.0 + Lenis 1.3.23 + GSAP 3.13+ (free incl. ScrollSmoother/SplitText/ScrambleTextPlugin/MorphSVG/MotionPath) + Motion 12.40 + visx 3.12 + ECharts 6.1 + Base UI 1.5 + Phosphor. Default Mol* to `(?illumination=1)`, `(plugin-`

`config.transparency=dpoit`, `enableAnimation=true`). Run all scenes pre-mounted via `<Activity mode="hidden">`. Ship.

Stage 2 (GPU compute). Add TSL compute shaders for the 1 M-particle closed-loop scene, the polymer cleavage animation, and GPU marching-cubes for live SES contour toggling. Move particle/physics work entirely to compute pipelines using `instancedArray()` + `Fn()` + `renderer.computeAsync()`.

Stage 3 (Path tracing + Gaussian splats). Add `@react-three/gpu-pathtracer` 0.3.x for converged hero stills, triggered when scroll velocity ≈ 0 (min 5 samples $\rightarrow$ 256 samples progressive). Capture and ship a `.splat` Gaussian splat of a real child's stretch garment for the dissolve scene; serve with COOP+COEP headers and `gpuAcceleratedSort: true`.

Stage 4 (AI co-pilot). Ship WebLLM (Qwen 2.5 1.5B INT4 or Phi-3.5-mini INT4) with progress-aware first-download UX + IndexedDB cache + WebGPU device check. Fall back to `@huggingface/transformers` v4 + WASM on unsupported devices. Pipe streaming output through ScrambleTextPlugin.

*Stage 5 (Mol customization).** Author a REWEAR-FUSED MVS namespace:

`StateTransformer.builderFactory('rewear-fused')` for a custom carbamate-bond representation (impostor cylinder with the cleavage progress uniform), a custom theme for "recyclability score" coloring, and a `MolstarLoadingExtension` to inject the polymer chain alongside the enzyme.

Thresholds that would change the plan.

- If WebGPU adapter fails for >10 % of the target audience in telemetry: prioritize the WebGL2 fallback path; pre-bake SES meshes; skip TSL compute particles in favor of pre-rendered video on those devices.
 - If first-paint LCP > 2.5 s: collapse the cinematic loader to a single TSL noise plane and defer Mol* until visible.
 - If model download budget < 200 MB on mobile: route the AI co-pilot to Chrome Built-in AI (Gemini Nano) when available, else a server endpoint, instead of WebLLM.
 - If Mol* picking/render perf drops below 60 fps with the dashboard's many mini-viewers: hoist all mini-canvases to a single Mol* canvas with multiple `Camera` snapshots, or pre-render mini-viewer thumbnails at build time using the new 8K UHD `ViewportScreenshotHelper`.
-

Caveats

- *Mol itself is still WebGL2** (with WebGL1 fallback) as of v5.9.0 — WebGPU is monitored but not shipped (the lead developer's Khronos talk and the WebGL Dev List exchange both list WebGPU as future work; molstar.org embeds the Web3DSurvey collector). This means *the Mol canvas is not GPU-shared with your `(three/webgpu)` canvas**; you will run two GL contexts on one device. Plan accordingly: keep one Mol* instance and reuse it across screens; do not spawn dozens of Mol* canvases.
- **Path tracing is not real-time** — converged stills require multiple seconds even on RTX-class GPUs. Use it as a "reward render" on scroll-stop, not as a default.
- **In-browser LLMs cap at ~2 B parameters for fluid mobile UX**; budget for a 500 MB–2 GB first-load download and an LRU IndexedDB cache. Frontier reasoning (long context, multi-step) still belongs server-side. The Hugging Face headline 60 tok/s on a 20 B model is specifically on Apple **M4 Pro Max**; integrated GPUs will be $\geq 5\times$ slower. (Hugging Face)
- **Awwwards-winning often = mobile-fragile**. Use device-detection to drop particle counts (e.g., 1 M $\rightarrow$ 50 K on mobile), reduce post-processing passes, and skip path tracing entirely on integrated GPUs.
- *The Mol SSGI is screen-space**, not full path tracing — it improves depth perception but does not converge to ground truth; for the hero still, use three-gpu-pathtracer on the parallel R3F canvas, not Mol*'s SSGI.
- **R3F v9 + WebGPU**: Drei components mostly work; the few that depend on WebGL-specific renderer features (some postprocessing wrappers, SSR via `@react-three/postprocessing`) need the TSL post-processing stack instead. Verify each Drei import before relying on it.
- **Server Components + R3F**: every Canvas must be a Client Component; the editorial shell can be Server, but the moment you import `three`, you've opted into Client. Use `dynamic(() => import(...), {ssr: false})` for the heavy canvases.
- **The molstar.org/mol-view-stories project (MolViewStories, Slanináková et al. 2026)** is excellent inspiration but is a separate web app — for REWEAR-FUSED, author your own MVS `.mvsj` files directly and load them via Mol*'s MVS extension `loadMVS()` rather than depending on the MolViewStories app. (GitHub) (Mol*)
- **The Nov 25 2025 WebGPU Baseline announcement is recent**; expect driver bugs especially on Linux (Firefox still working through it in 2026), older Intel integrated GPUs, and Samsung Xclipse Android GPUs (WIP per the W3C implementation-status wiki). Telemetry-driven progressive enhancement remains mandatory. (GitHub)
- **Stage 1 alone already exceeds what most 2026 Awwwards Site of the Year winners ship**. Stages 2–5 are the "shouldn't be possible on the web" delta. The 72-hour window is explicitly not a factor per the brief — but each stage has a clean cut-off, so the project remains shippable at any of the five gates.

